

Explaining R-GCN Node Classification on the AIFB RDF Knowledge Graph

Steevan Gonsalves (4089415), Ashton Prince Rodrigues (4107301), and Aman Sami Ur Rahman Mohammed (4107422)

Paderborn University

Abstract. This mini-project studies explainable node classification on an RDF knowledge graph. We use the AIFB RDF dataset, convert RDF triples into a PyTorch Geometric graph, train a two-layer Relational Graph Convolutional Network (R-GCN) using **FastRGCNConv**, and explain one selected prediction with GNNExplainer. The task is to classify labeled person nodes into affiliation classes using graph structure, relation types, and degree-based node features. The final model achieves a test accuracy of 94.44% on a deterministic 80/20 stratified train-test split. GNNExplainer identifies important RDF relationships involving research groups, projects, publications, research areas, and people. The explanation is converted back into human-readable RDF triples so that the model decision can be interpreted in the original knowledge graph domain.

Keywords: Explainable AI · RDF · Knowledge Graphs · R-GCN · GNNExplainer · PyTorch Geometric

1 Introduction

Graph neural networks are commonly used for learning on structured data such as social networks, citation networks, molecular graphs, and knowledge graphs [3, 14]. RDF knowledge graphs are especially interesting because their edges have semantic relation types, for example *person member project* or *research group publishes publication*. This makes RDF data suitable for relational graph neural networks, but it also creates a need for explanation methods that can show which graph relationships influenced a prediction.

The goal of this project is to explain the predictions of a machine learning model on an RDF dataset, following the course motivation that XAI should make model behavior more understandable to users [12]. We use the AIFB RDF Knowledge Graph as the dataset and formulate the problem as node classification. The model predicts the affiliation class of person entities. The project follows the complete workflow required for the mini-project: selecting an RDF dataset, analyzing the data, training a model, evaluating predictive performance, generating explanations, and interpreting the explanations [11, 10].

For the predictive model, we use a two-layer R-GCN implemented with PyTorch Geometric’s **FastRGCNConv** [1, 3, 5]. R-GCNs are appropriate for RDF

graphs because they can use different relation types during message passing. For explanation, we use GNNExplainer, which learns an edge importance mask for a selected prediction [2, 6]. The explanation is then converted back from numerical edge indices into RDF triples, making the result understandable in terms of entities and relations from the original dataset.

The project does not aim to introduce a new graph learning algorithm. Instead, it applies existing graph neural network and explanation methods to an RDF dataset and focuses on making the full pipeline reproducible and interpretable.

2 Data Analysis

2.1 Dataset

The selected dataset is the AIFB RDF Knowledge Graph [9]. It describes academic entities such as people, research groups, publications, projects, and research areas. The RDF graph is stored in the file `aifbfixed_complete.n3`. Classification labels are loaded from `completeDataset.tsv`.

Each RDF triple has the form:

`Subject -> Predicate -> Object`

In the graph conversion, subjects and objects become graph nodes, while predicates become relation types. A directed edge is created for every RDF triple.

2.2 Dataset Statistics

The RDF graph contains 29,226 triples. After converting the RDF graph into a PyTorch Geometric graph, the graph contains 8,285 nodes and 47 relation types. The classification file contains 176 labeled person nodes and 4 affiliation classes. All other graph nodes are unlabeled and are assigned the label value -1.

Table 1. AIFB dataset statistics used in this project.

Statistic	Value
RDF triples	29,226
Graph nodes	8,285
Relation types	47
Labeled nodes	176
Unlabeled nodes	8,109
Target classes	4
Node features after preprocessing	72

2.3 Graph Conversion

The RDF graph is loaded with RDFLib [7]. Unique subjects and objects are collected to form the set of graph nodes. Predicates are collected as relation types. For reproducibility, nodes, relations, and RDF triples are sorted before assigning numerical IDs. This is important because unordered Python sets may assign different node indices after a kernel restart.

The final PyTorch Geometric data object contains:

- **x**: node feature matrix,
- **edge_index**: directed graph edges,
- **edge_type**: relation type of each edge,
- **y**: node labels,
- **train_mask**: training nodes,
- **test_mask**: test nodes.

2.4 Node Features and Labels

Node features are created using PyTorch Geometric’s **OneHotDegree** transform [5]. This produces degree-based structural features. Empty feature columns are removed, resulting in a final feature matrix of shape:

8285 x 72

Labels are available only for 176 person nodes. Unlabeled nodes remain in the graph because they provide useful graph structure for message passing, but they are never used in the supervised training loss. This is verified in the notebook using assertions that ensure the training and test masks contain only labeled nodes.

3 Model Training and Evaluation

3.1 Train-Test Split

The 176 labeled nodes are split into training and test sets using an 80/20 split. Stratified sampling is used to preserve the class distribution. The split is deterministic because a fixed random seed is used. The implementation uses scikit-learn’s **train_test_split** utility [8].

- Training nodes: 140
- Test nodes: 36

Only labeled nodes are included in the split. The notebook verifies that **train_mask** and **test_mask** do not include unlabeled nodes and do not overlap.

3.2 Model Architecture

The model is a two-layer Relational Graph Convolutional Network. The implementation uses `FastRGCNConv` from PyTorch Geometric [1, 5]. The architecture is:

```
FastRGCNConv(72 -> 16) -> ReLU -> FastRGCNConv(16 -> 4)
```

The input dimension is 72 because of the generated degree-based node features. The hidden layer has 16 units, and the output layer has 4 units corresponding to the four affiliation classes.

3.3 Training Setup

The model is trained for 100 epochs with the Adam optimizer and a learning rate of 0.01, implemented with PyTorch [4]. Cross-entropy loss is computed only on training nodes:

```
F.cross_entropy(out[data.train_mask], data.y[data.train_mask])
```

This ensures that unlabeled nodes are not included in the supervised loss. They only contribute indirectly through the graph structure during message passing.

3.4 Evaluation Results

The final model achieves a training accuracy of 97.86% and a test accuracy of 94.44%. Out of 36 test nodes, 34 are classified correctly.

Table 2. Final model performance.

Metric	Value
Training accuracy	97.86%
Test accuracy	94.44%
Correct test nodes	34 / 36

The test accuracy briefly reaches 97.22% during training and later stabilizes at 94.44%. This kind of fluctuation is expected because the test set contains only 36 nodes. A single changed prediction changes the test accuracy by approximately 2.78 percentage points.

3.5 Reproducibility

The notebook was updated to make the experiment as reproducible as possible. A single seed is used for Python, NumPy, PyTorch, model initialization, train-test splitting, and GNNExplainer initialization. In addition, RDF nodes, relations, and triples are sorted before numerical IDs are assigned.

After these changes, restarting the kernel and running all cells produces the same final accuracy and selected explanation node in the tested environment. Small differences may still occur across different hardware, PyTorch versions, PyTorch Geometric versions, or CUDA backends because some low-level operations may be nondeterministic.

4 Model Explanation

4.1 Explanation Method

To explain the trained R-GCN, we use GNNExplainer [2, 6]. GNNExplainer learns an edge mask that assigns an importance score to each edge in the graph. Higher scores indicate that the edge was more influential for the selected prediction. The use of local explanation methods follows the course discussion of explaining individual predictions [13, 14].

The explainer is configured for node-level multiclass classification. We explain the model prediction rather than the ground-truth label. Node feature masks are disabled, and edge masks are enabled. This choice focuses the explanation on RDF relationships, which is appropriate because the purpose of the project is to understand which graph relationships influenced the prediction.

4.2 Selected Node

The selected node is chosen from the correctly classified test nodes. Among those nodes, the node with the highest prediction confidence is selected for explanation.

Table 3. Selected node for GNNExplainer.

Property	Value
Selected node index	5710
Predicted class	2
True class	2
Prediction confidence	1.0

This node is suitable for explanation because the prediction is correct and the model is highly confident.

4.3 Important Edges

GNNExplainer returns an importance score for each edge. The top 10 edge importance scores are:

[0.8011, 0.7937, 0.7818, 0.7707, 0.7613, 0.7464, 0.1365, 0.1365,
0.1362, 0.1362]

The corresponding edge indices are:

[2095, 9069, 21478, 1480, 2574, 1455, 5928, 25052, 27301, 15586]

The first six edges receive substantially higher scores than the remaining four. This suggests that the explanation is mainly driven by a small group of important RDF relationships.

4.4 Human-Readable RDF Interpretation

To make the explanation interpretable, the top-ranked edge indices are converted back into RDF triples. The most influential relationships are:

- Research Group -> carriesOut -> Project
- Project -> member -> Person
- Publication -> editor -> Person
- Research Area -> isWorkedOnBy -> Person
- Research Group -> publishes -> Publication
- Research Area -> dealtWithIn -> Project

These relationships are meaningful for the classification task. The model predicts a person’s affiliation, and information about research groups, projects, publications, and research areas can provide evidence about that affiliation. For example, a project can be connected to a person through membership, a research group can be connected to projects and publications, and research areas can connect people to academic topics.

The lower-scoring edges provide additional supporting information, such as:

- Person -> publication -> Publication
- Publication -> isAbout -> Research Area

These edges receive lower importance scores, but they still support the final explanation by connecting the selected person to publications and research areas.

4.5 Evaluation of the Explanation

The explanation is reasonable because the highest-scoring edges correspond to academic structures that are relevant to affiliation prediction. The explanation is not only numerical; it can be inspected in the original RDF domain. This is important because edge scores alone do not explain the semantic meaning of the model decision.

The explanation also matches the dataset context. The AIFB graph contains people, projects, publications, research groups, and research areas. The top-ranked edges connect exactly these kinds of entities. Therefore, the explanation suggests that the R-GCN uses meaningful relational evidence rather than arbitrary graph edges.

At the same time, the explanation should be interpreted as a local explanation for one selected node, not as a complete global explanation of the whole model. GNNExplainer optimizes an explanation mask for a specific prediction. The important edges for another test node may be different.

5 Conclusion

This project demonstrates a complete explainable AI workflow on an RDF knowledge graph. The AIFB dataset was loaded and analyzed, converted into a PyTorch Geometric graph, used to train a two-layer R-GCN, and evaluated on a deterministic train-test split. The final model achieved 94.44% test accuracy. GNNExplainer was then used to explain one correct and confident prediction. By converting important edge indices back into RDF triples, the explanation became human-readable and meaningful in the original knowledge graph domain.

6 Contributions of Team Members

All team members contributed to the project implementation, analysis, and documentation. Steevan Gonsalves worked on RDF data loading, graph conversion, and reproducibility improvements. Ashton Prince Rodrigues worked on R-GCN model training, evaluation, and result analysis. Aman Sami Ur Rahman Mohammed worked on GNNExplainer configuration, RDF triple interpretation, and report preparation. The final notebook and report were reviewed collaboratively to ensure that the project satisfies the mini-project requirements.

ChatGPT was used as an assistance tool for coding problem solving, debugging, improving reproducibility, and understanding implementation issues. The final technical decisions, notebook execution, result verification, and report content were reviewed by the team [15].

References

1. Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling Relational Data with Graph Convolutional Networks. In: European Semantic Web Conference, pp. 593–607 (2018)

2. Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: Generating Explanations for Graph Neural Networks. In: Advances in Neural Information Processing Systems (2019)
3. Fey, M., Lenssen, J.E.: Fast Graph Representation Learning with PyTorch Geometric. In: ICLR Workshop on Representation Learning on Graphs and Manifolds (2019)
4. PyTorch contributors: PyTorch documentation. <https://pytorch.org/docs/stable/index.html>
5. PyTorch Geometric contributors: PyTorch Geometric documentation. <https://pytorch-geometric.readthedocs.io/>
6. PyTorch Geometric contributors: PyTorch Geometric explainability documentation. <https://pytorch-geometric.readthedocs.io/en/latest/modules/explain.html>
7. RDFLib contributors: RDFLib documentation. <https://rdflib.readthedocs.io/>
8. scikit-learn developers: `train_test_split` documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
9. AIFB RDF dataset. <https://www.aifb.kit.edu/>
10. Heindorf, S.: Explainable AI, Organization. Course lecture slides, Paderborn University (2026).
11. Heindorf, S.: Explainable AI, Mini Project. Course lecture slides, Paderborn University (2026).
12. Heindorf, S.: Explainable AI, Part 01: Introduction. Course lecture slides, Paderborn University (2026).
13. Heindorf, S.: Explainable AI, Part 05: Local Model-Agnostic Explanations. Course lecture slides, Paderborn University (2026).
14. Heindorf, S.: Explainable AI, Part 07: Neural Networks. Course lecture slides, Paderborn University (2026).
15. OpenAI: ChatGPT. Used for coding problem solving, debugging, reproducibility improvements, and support in understanding implementation issues. <https://chat.openai.com/>